Checklist:

# The Pragmatic Tech Lead

## Pragmatic Tech Lead Checklist

### 1. Clarify the tech lead role

- ☐ I understand whether “tech lead” is a formal title, an informal role, or a project-specific responsibility.
- ☐ I know what my manager expects from me as tech lead.
- ☐ I know where I should spend my time across:
  - ☐ Building software
  - ☐ Strategy and alignment
  - ☐ Helping people and teams grow
- ☐ I am still hands-on enough to understand the system and set a quality bar.
- ☐ I am not acting like a manager unless that responsibility is explicitly mine.
- ☐ I lead by example: planning, writing code, reviewing code, supporting releases, and joining on-call where appropriate.
- ☐ I avoid becoming a bottleneck where everyone waits for me to make a decision.

## Project Leadership Checklist

### 2. Start the project well

- ☐ The project has a clear business goal.
- ☐ The problem being solved is explicit.
- ☐ The expected outcome is clear.
- ☐ The project scope is written down.
- ☐ The project has an owner or project lead.
- ☐ The project has a kickoff before major work starts.
- ☐ The kickoff aligns stakeholders on:

    - [ ] Why we are doing this
    - [ ] What we are building
    - [ ] How we plan to build it
    - [ ] When we expect milestones
- [ ] The proposal or PRD is circulated before the kickoff.
- [ ] Product, engineering, design, data, business, and other relevant stakeholders have reviewed the plan.
- [ ] Open questions are captured before implementation starts.

### 3. Run an engineering kickoff

- [ ] The engineering team understands the product and business goal.
- [ ] The team has discussed the technical approach.
- [ ] Architecture, APIs, data, infrastructure, and operational concerns are covered.
- [ ] Important decisions are written down.
- [ ] RFCs, ERDs, ADRs, or similar documents are created where useful.
- [ ] Relevant engineering teams are included early.
- [ ] The team understands dependencies and constraints.
- [ ] The plan is available for future onboarding and reference.

### 4. Establish milestones and estimates

- [ ] The project is broken into small, shippable milestones.
- [ ] Milestones are small enough to validate progress every few weeks or sooner.
- [ ] Estimates are treated as rough forecasts, not fixed commitments.
- [ ] The team understands what uncertainty remains.
- [ ] Risks that could change the estimate are visible.
- [ ] Stakeholders understand that dates become more reliable as milestones are reached.
- [ ] The team avoids locking in dates before engineering planning is complete.
- [ ] If the business needs a fixed date, scope and quality tradeoffs are discussed explicitly.

## Scope, Timeline, and People Checklist

### 5. Manage “software project physics”

- [ ] Scope, timeline, and people are considered together.
- [ ] When scope increases, we adjust at least one of:
    - [ ] Timeline
    - [ ] Staffing
    - [ ] Quality expectations
    - [ ] Scope elsewhere
- [ ] When the timeline shrinks, we realistically reduce scope or increase capacity.
- [ ] When fewer people are available, we reduce scope or extend the timeline.
- [ ] When adding people, we account for onboarding and coordination costs.
- [ ] We avoid assuming that adding people will automatically speed up delivery.
- [ ] We communicate tradeoffs clearly to stakeholders.
- [ ] We do not hide scope, timeline, or people changes until they become crises.

# Day-to-Day Project Management Checklist

## 6. Keep the project moving

- [ ] Everyone understands the current project status.
- [ ] Everyone knows the next milestone.
- [ ] Everyone knows who owns each major workstream.
- [ ] Blockers are visible.
- [ ] Decisions are written down.
- [ ] The team has a lightweight way to track progress.
- [ ] The project management approach fits the team, rather than blindly following a framework.
- [ ] The team inspects and adapts its process.
- [ ] We avoid process for its own sake.
- [ ] We keep the ultimate business goal visible.

## 7. Make decisions responsibly

- [ ] I know which decisions I can make independently.
- [ ] I know which decisions require consultation.
- [ ] I consult the people closest to the information before making a decision.
- [ ] I inform stakeholders when a decision affects scope, timeline, people, quality, or risk.
- [ ] I share:

    - [ ] The situation or problem
    - [ ] The options considered
    - [ ] The tradeoffs
    - [ ] The recommended approach
- [ ] I avoid surprising stakeholders with major changes.
- [ ] I become more predictable over time by consistently using good judgment.

# Risk and Dependency Checklist

## 8. Identify major risks

- [ ] Technology risks are identified.
- [ ] Engineering dependencies are identified.
- [ ] Non-engineering dependencies are identified.
- [ ] Missing decisions or unclear context are identified.
- [ ] Unrealistic timelines are challenged.
- [ ] Bandwidth or staffing risks are visible.
- [ ] Surprises discovered mid-project are reassessed.
- [ ] Unknown estimate confidence is communicated.

## 9. Mitigate technology risk

- [ ] We prototype unfamiliar technology.
- [ ] We review roadmap, maintenance, and support risks.
- [ ] We avoid unstable tools unless the risk is justified.
- [ ] We consider simpler or more reliable alternatives.
- [ ] We split unknown work into smaller explorations or spikes.

## 10. Mitigate dependency risk

- [ ] Upstream teams know what we need from them.
- [ ] Downstream teams know what changes may affect them.
- [ ] We talk directly to teams we depend on.
- [ ] We offer to do work ourselves where appropriate.
- [ ] We mock or stub dependencies when useful.
- [ ] We remove critical-path dependencies where feasible.
- [ ] We escalate only after trying engineer-to-engineer communication.

- [ ] We do not start work when the “why” or “what” is still unclear.

# Wrapping Up Projects Checklist

## 11. Close projects properly

- [ ] “Done” is clearly defined.
- [ ] The project has a final update.
- [ ] The final update is written soon after the project team disbands.
- [ ] The update explains:
    - [ ] What shipped
    - [ ] What changed
    - [ ] What impact was achieved
    - [ ] What was learned
    - [ ] What remains open
- [ ] Stakeholders are thanked.
- [ ] Contributors are recognized.
- [ ] The team celebrates the project.
- [ ] A retrospective is held where useful.
- [ ] Learnings are captured for future projects.
- [ ] Even failed or partially successful projects are wrapped up constructively.

# Shipping to Production Checklist

## 12. Choose an appropriate release approach

- [ ] The release process matches the product's risk.
- [ ] We know whether this is closer to:
    - [ ] YOLO shipping
    - [ ] Startup-style release
    - [ ] Traditional QA-heavy release
    - [ ] Large-tech automated release
    - [ ] Highly regulated release
- [ ] The team understands the cost of bugs in production.
- [ ] The team understands how quickly it needs feedback.

- [ ] We are not overbuilding the release process unnecessarily.
- [ ] We are not under-protecting users or the business.

## 13. Verify changes before release

- [ ] Code is reviewed.
- [ ] Automated tests run.
- [ ] Edge cases are considered.
- [ ] Local or isolated verification is performed.
- [ ] CI/CD checks are passing.
- [ ] Expensive tests are run where needed.
- [ ] Monitoring is ready before rollout.
- [ ] On-call or support ownership is clear.
- [ ] Runbooks or mitigation steps exist for likely failures.

## 14. Use safety nets where appropriate

- [ ] Separate test or staging environments are used when useful.
- [ ] QA or exploratory testing is involved where appropriate.
- [ ] Feature flags are used for risky or gradual changes.
- [ ] Canary releases are used when production validation is needed.
- [ ] Staged rollouts are planned for risky changes.
- [ ] Automated rollback is available where possible.
- [ ] Rollback steps are easy to execute.
- [ ] Metrics define whether rollout should continue.
- [ ] Customer feedback is monitored after release.
- [ ] Incidents are handled blamelessly.

## 15. Take pragmatic risks consciously

- [ ] Any bypassed process is intentionally bypassed, not ignored accidentally.
- [ ] Risky changes are communicated to relevant stakeholders.
- [ ] There is a rollback plan.
- [ ] Customer support and on-call teams know what may happen.
- [ ] We inspect customer feedback after release.
- [ ] We track incidents and measure their impact.
- [ ] Error budgets or similar risk limits guide how much risk is acceptable.

# Stakeholder Management Checklist

## 16. Identify stakeholders

- [ ] Customers or users are considered.
- [ ] Business stakeholders are identified.
- [ ] Product stakeholders are identified.
- [ ] Engineering stakeholders are identified.
- [ ] External stakeholders are identified where relevant.
- [ ] Upstream dependencies are identified.
- [ ] Downstream dependencies are identified.
- [ ] Strategic stakeholders are identified.
- [ ] “Usual suspects” are checked:
    - [ ] Engineering
    - [ ] Product
    - [ ] Design / UX
    - [ ] Data science
    - [ ] Security and compliance
    - [ ] Infrastructure / DevOps / SRE
    - [ ] Legal
    - [ ] Marketing/growth
    - [ ] Customer support
    - [ ] Operations
    - [ ] Sales/business development
    - [ ] Finance
- [ ] Existing stakeholders are asked who else should be involved.
- [ ] Architecture and code ownership are reviewed to find hidden stakeholders.

## 17. Keep stakeholders informed

- [ ] Stakeholders know the project goal.
- [ ] Stakeholders know the current status.
- [ ] Stakeholders know the major risks.
- [ ] Stakeholders know timeline changes.
- [ ] Stakeholders know when their input is needed.
- [ ] Updates are sent regularly.
- [ ] Updates are clear enough to become a shared source of truth.
- [ ] Meetings are used when discussion is needed.
- [ ] Async updates are used when information sharing is enough.

- [ ] Hybrid communication is used for important changes.
- [ ] Stakeholders are not surprised late in the project.

### 18. Handle problematic stakeholders

- [ ] I distinguish between upstream, downstream, and strategic stakeholder problems.
- [ ] I talk face-to-face or via video when written communication fails.
- [ ] I explain what needs to happen, why, and where we are now.
- [ ] I educate impatient stakeholders with transparent updates.
- [ ] I ask for management or leadership support when direct communication fails.
- [ ] I avoid treating stakeholder management as a bureaucratic process; the goal is project success.

### 19. Learn from stakeholders

- [ ] I ask stakeholders about their part of the business.
- [ ] I ask what challenges they face.
- [ ] I ask what they do outside this project.
- [ ] I build relationships before crises.
- [ ] I shadow or observe other teams where useful.
- [ ] I learn enough about stakeholder domains to communicate in their language.

## Team Structure Checklist

### 20. Clarify roles and responsibilities

- [ ] Titles and roles are not confused.
- [ ] Temporary project roles are explicit where needed.
- [ ] The project lead is clear.
- [ ] Support or rotating responsibilities are clear.
- [ ] On-call ownership is clear.
- [ ] Stakeholder point-of-contact is clear.
- [ ] Team members know who owns what.
- [ ] Role overlaps are identified.
- [ ] Role gaps are identified.
- [ ] Implicit roles become explicit when confusion arises.

### 21. Maintain useful team processes

- [ ] Planning processes help the team move faster or deliver higher-quality work.
- [ ] Building processes support delivery.
- [ ] Releasing processes reduce risk.
- [ ] Maintaining processes support reliability and ownership.
- [ ] Productivity processes remove friction.
- [ ] Team health processes improve morale and collaboration.
- [ ] Redundant processes are removed.
- [ ] Time-consuming manual processes are candidates for automation.
- [ ] The team periodically asks: “Does this process still help?”

### 22. Boost team focus

- [ ] The team knows its current top priorities.
- [ ] The team knows why those priorities matter.
- [ ] Priorities are written down and confirmed with management.
- [ ] I remind the team of the current focus regularly.
- [ ] Sudden priority changes are challenged respectfully.
- [ ] New work has a clear impact.
- [ ] New work has a written spec or clear “why” and “what.”
- [ ] Engineering feasibility is assessed before committing.
- [ ] Context-switching cost is made explicit.
- [ ] Alternatives are offered before disrupting the whole team.
- [ ] I protect the team from vague, urgent, low-value interruptions.

## Team Dynamics Checklist

### 23. Assess team health

- [ ] The team has clarity.
- [ ] The team executes and ships.
- [ ] Morale is good.
- [ ] Communication is respectful and constructive.
- [ ] People feel psychologically safe.
- [ ] Team members are engaged.
- [ ] Junior and new members are invited to participate.

- [ ] People are comfortable raising concerns.
- [ ] Good work is recognized.
- [ ] The team has healthy relationships with nearby teams.

## 24. Watch for unhealthy patterns

- [ ] Lack of clarity is visible.
- [ ] Poor execution is addressed.
- [ ] Conflict is handled constructively.
- [ ] Communication breakdowns are noticed.
- [ ] Feedback is specific and useful.
- [ ] Psychological safety is protected.
- [ ] Everyone contributes.
- [ ] Too much process is reduced.
- [ ] Too little structure is corrected.
- [ ] Tech debt is not ignored.
- [ ] Excessive context switching is reduced.
- [ ] The team is not stuck “treading water.”

## 25. Address team growing pains

- [ ] Silent conflicts or cliques are noticed.
- [ ] Execution problems are addressed before they become crises.
- [ ] Good work outside the team is recognized.
- [ ] Attrition risk is handled carefully and empathetically.
- [ ] Senior-heavy teams have enough meaningful challenges.
- [ ] Junior-heavy teams get enough support and review.
- [ ] New joiners are onboarded without overloading the team.
- [ ] Direction changes are communicated as temporary disruption, not failure.

## 26. Improve team dynamics

- [ ] I observe before trying to fix things.
- [ ] I speak with team members privately to understand issues.
- [ ] I reduce negative interactions in group settings.
- [ ] I involve team members in relevant decisions.
- [ ] I escalate serious problems to the manager when needed.
- [ ] I suggest solutions, not just problems.
- [ ] I avoid surprising my manager with major team issues.

- [ ] I use my influence to help the team become healthier, not to dominate it.

### 27. Build relationships with other teams

- [ ] I regularly catch up with engineering leads on other teams.
- [ ] I know managers or product managers on teams we depend on.
- [ ] I use verbal communication for problematic cross-team situations.
- [ ] I avoid letting written channels amplify misunderstanding.
- [ ] I share what is happening on my team.
- [ ] I offer help to other teams where possible.
- [ ] I build relationships before dependencies become urgent.

## Final Tech Lead Takeaways

- [ ] I balance project leadership, technical contribution, stakeholder alignment, and team support.
- [ ] I keep stakeholders, product, and my manager in the loop.
- [ ] I empower team members instead of centralizing all decisions.
- [ ] I remove blockers.
- [ ] I protect quality by leading through example.
- [ ] I do not act superior to other engineers.
- [ ] I create an environment where engineers have voice, confidence, and ownership.
- [ ] I help the team and project succeed, rather than making the team dependent on me.